

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T08:47:47.0960633Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-084747212.prof
Profil mémoire	memory-20260924-084747212.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	6.69 ms	6685758	44966	17

Détail hyperfine

Métrique	Valeur
Moyenne	4.6804 s
Écart-type	n/a
Médiane	4.6804 s
Temps utilisateur	20.5081 s
Temps système	2.1025 s
Min	4.6804 s
Max	4.6804 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	20.79s
Mémoire allouée (pprof)	82.48MB
Mémoire par opération	44966 B/op

Garbage collector Go

Cycles GC	64
Temps GC cumulé	74.790 ms

Plus longue pause	4.000 ms
Temps CPU GC cumulé	465.190 ms
Objectif mémoire maximal	95.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **20.79s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).infectionCandidate	17.31s	83.26%	19.48s	93.70%
jeu-de-la-vie/game.(*Board).index (inline)	1.75s	8.42%	1.77s	8.51%
jeu-de-la-vie/game.(*Board).Step	0.58s	2.79%	1s	4.81%
runtime.asyncPreempt	0.26s	1.25%	0.26s	1.25%
jeu-de-la-vie/game.(*Board).countInfectionCand	0.20s	0.96%	19.68s	94.66%
math/rand.(*Rand).Float64 (inline)	0.19s	0.91%	0.37s	1.78%
jeu-de-la-vie/game.squaredDistance (inline)	0.16s	0.77%	0.16s	0.77%
math/rand.(*Rand).Int63 (inline)	0.06s	0.29%	0.18s	0.87%

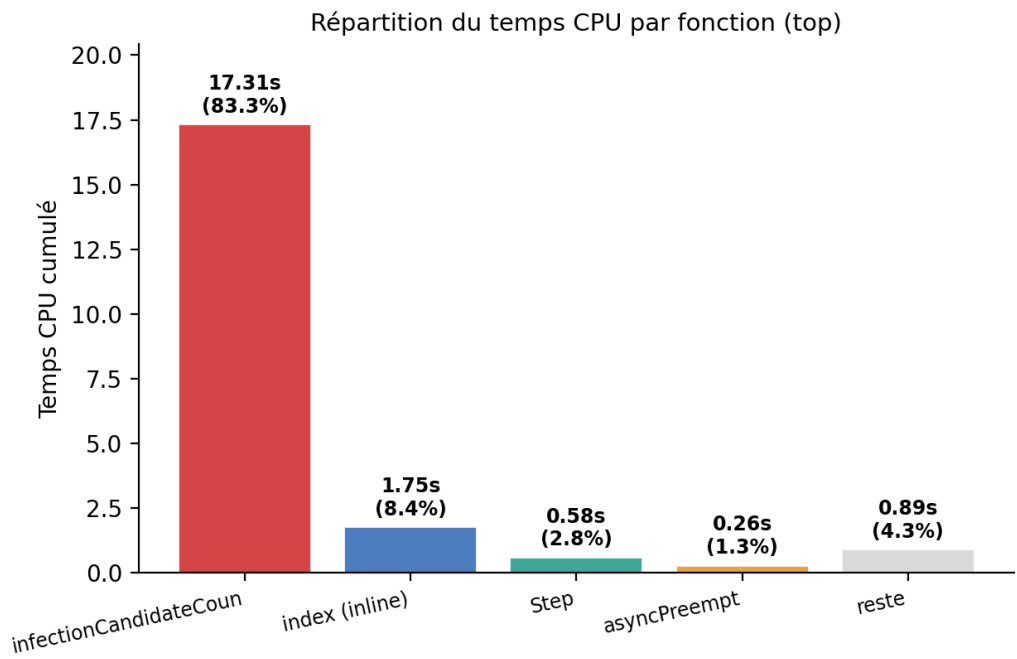


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

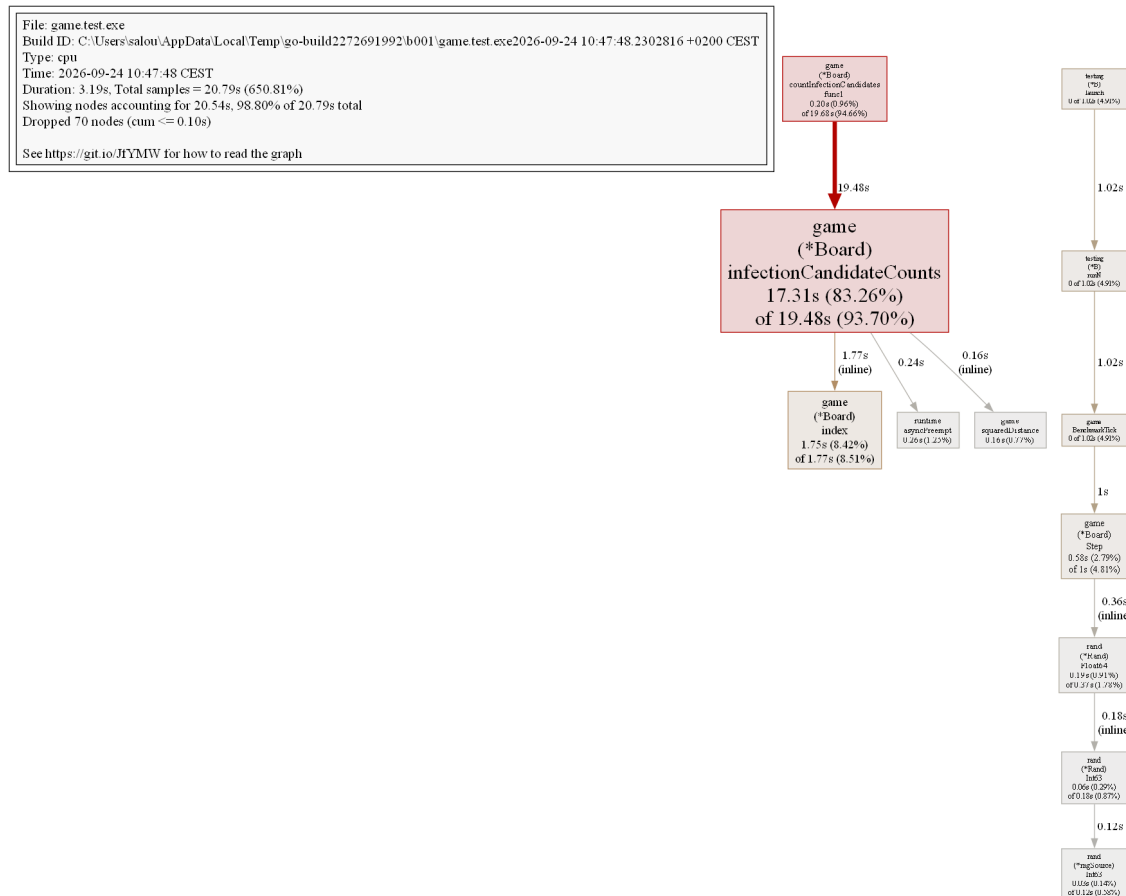


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).infectionCandidateCounts**, responsable de **83.3%** du temps CPU total (flat), pour **93.7%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

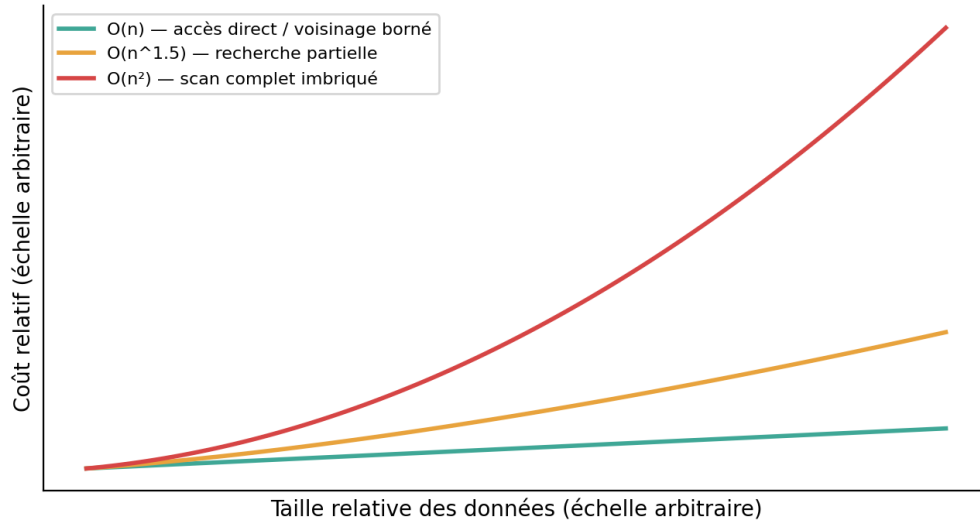


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **82.48MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).ensureScratch (inl	40.73MB	49.38%	40.73MB	49.38%
jeu-de-la-vie/game.NewBoard (inline)	16.57MB	20.09%	16.57MB	20.09%
jeu-de-la-vie/game.RandomBoard	13.81MB	16.74%	30.38MB	36.83%
runtime.mallocgc	3.51MB	4.25%	3.51MB	4.25%
jeu-de-la-vie/game.(*Board).countInfectionCand	1.50MB	1.82%	1.50MB	1.82%
runtime/pprof.StartCPUProfile	1.16MB	1.40%	1.16MB	1.40%
syscall.UTF16ToString	1MB	1.21%	1MB	1.21%
compress/flate.(*compressor).init	0.64MB	0.77%	1.71MB	2.07%

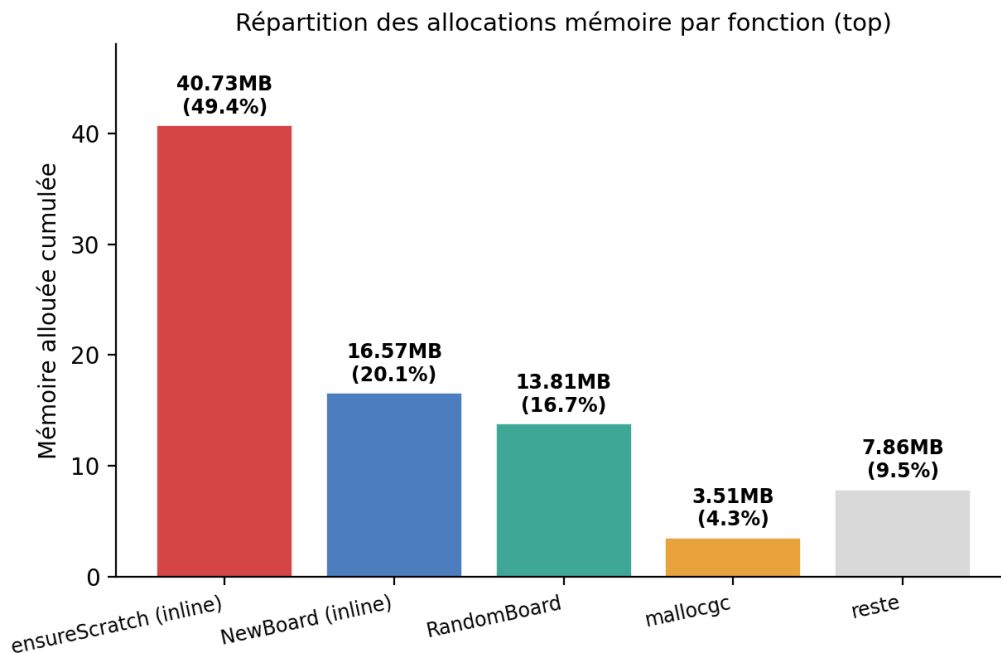


Figure — Répartition des allocations mémoire par fonction.

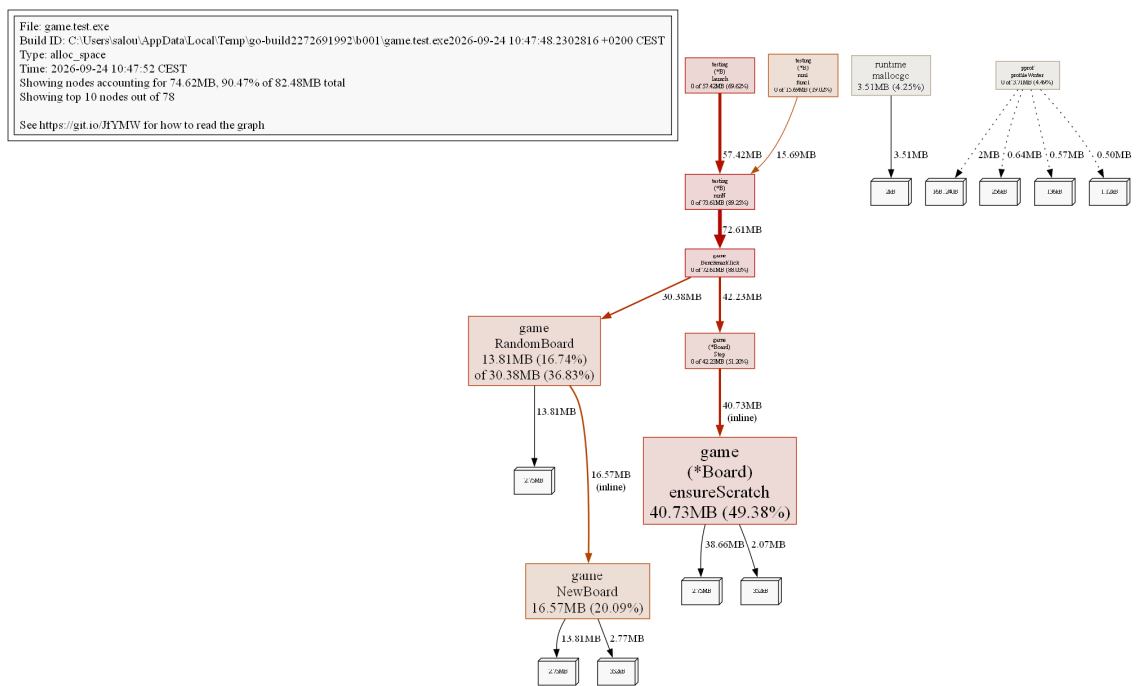


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 10:48:01 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.